

Week 6

2. Code Completion

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Wednesday, February 11th, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Agenda



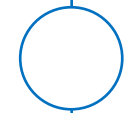
APIs in Java:

- **About APIs**
- **Code Completion Techniques**
- **Context-Sensitive Code Completion (CSCC)**
- **Conclusions**

Part I

Application Programming Interfaces (APIs)



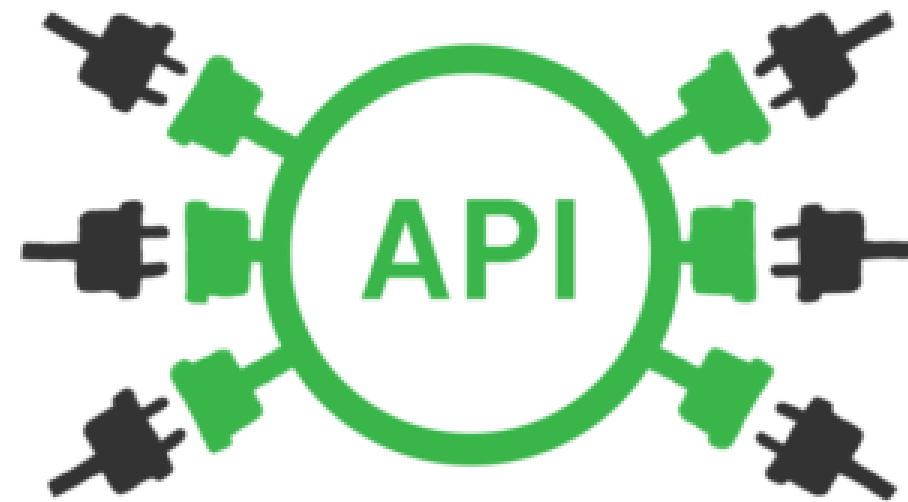


About APIs



API (APPLICATION PROGRAMMING INTERFACE)

- An API is the interface to implemented functionality that developers access to perform various tasks
[Robillard, IEEE Software, 2009]



WHY IS IT IMPORTANT TO DISCOVER APIs?

- Large in sizes and thus difficult to remember
- Complex relationship



WHY IS IT IMPORTANT TO DISCOVER APIs?

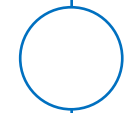
- How Much Information Do I Need to Remember ???
 - It is hard to remember every detail



WHY IS IT IMPORTANT TO DISCOVER APIs?

- It is hard to remember every detail
 - Besides its own methods, JFrame inherited a large number of methods





Code Completion



CODE COMPLETION IS THE KEY !

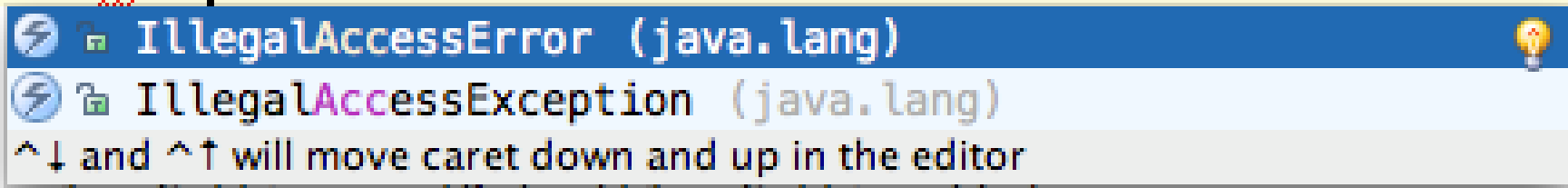
- One of the top ten commands executed by most developers
[Murphy et al., IEEE Software, 2006]

```
package com.jetbrains.samples.completion.dataflow;

import java.io.InputStream;

public class JavaSettingsApp {
    public static void main(String[] args) {
        if (args.length == 1) {
            String projectName = args[0];
            Project project = ProjectUtils.findProject(projectName);
            JavaSettings settings = project instanceof JavaProject ?
                ((JavaProject) project).getJavaSettings() : getDefaultJavaSet
            System.out.println(settings);
        } else {
            throw new acc
        }
    }

    private static JavaSettings getDefaultJavaSettings() {
        return ProjectUtils.getDefaultJavaSettings();
    }
}
```



BENEFITS



Helps to write compilable code



Speeds up the typing



Serves as a convenient
documentation

COMPONENTS

- Trigger

Trigger:
Typing the dot (.) after the padding variable activates the code completion system). Dot is the trigger character here.

Algorithm:
All methods of StringBuffer receiver type are collected and sorted alphabetically. This constitutes the algorithm

Presentation:
1. a popup menu with method names
2. Another window that shows documentation of the selected method

Code Context:
Type of the padding variable

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("");
    for(int i=0;i<remainder;i++){
        padding.append("");
    }
    input = input+ padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```

append
public [StringBuffer](#) append(boolean b)

Appends the string representation of the boolean argument to the sequence.
The overall effect is exactly as if the argument were converted to a string by the method [String.valueOf\(boolean\)](#), and the characters of that string were then [appended](#) to this character sequence.

Parameters:
b - a boolean.

Returns:
a reference to this object

Press 'Tab' from proposal table or click for focus

Target: Method Call
When a developer selects a completion proposal from the popup menu, the completion system inserts the method call in the code

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("");
    for(int i=0;i<remainder;i++){
        padding.append("");
    }
    input = input+ padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```


COMPONENTS

- Trigger
- Algorithm

Trigger:
Typing the dot (.) after the padding variable activates the code completion system). Dot is the trigger character here.

Algorithm:
All methods of StringBuffer receiver type are collected and sorted alphabetically. This constitutes the algorithm

Presentation:
1. a popup menu with method names
2. Another window that shows documentation of the selected method

Code Context:
Type of the padding variable

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("");
    for(int i=0;i<remainder;i++){
        padding.append("");
    }
    input = input+ padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```

append
public [StringBuffer](#) append(boolean b)
Appends the string representation of the boolean argument to the sequence.
The overall effect is exactly as if the argument were converted to a string by the method [String.valueOf\(boolean\)](#), and the characters of that string were then [appended](#) to this character sequence.
Parameters:
b - a boolean.
Returns:
a reference to this object.
Press 'Tab' from proposal table or click for focus

Target: Method Call
When a developer selects a completion proposal from the popup menu, the completion system inserts the method call in the code

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("");
    for(int i=0;i<remainder;i++){
        padding.append("");
    }
    input = input+ padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```


COMPONENTS

- Trigger
- Algorithm
- Context

Trigger:
Typing the dot (.) after the padding variable activates the code completion system). Dot is the trigger character here.

Algorithm:
All methods of StringBuffer receiver type are collected and sorted alphabetically. This constitutes the algorithm

Presentation:
1. a popup menu with method names
2. Another window that shows documentation of the selected method

Code Context:
Type of the padding variable

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("");
    for(int i=0;i<remainder;i++){
        padding.append("");
    }
    input = input+padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```

append
public [StringBuffer](#) append(boolean b)
Appends the string representation of the boolean argument to the sequence.
The overall effect is exactly as if the argument were converted to a string by the method [String.valueOf\(boolean\)](#), and the characters of that string were then [appended](#) to this character sequence.
Parameters:
b - a boolean.
Returns:
a reference to this object.
Press 'Tab' from proposal table or click for focus

Target: Method Call
When a developer selects a completion proposal from the popup menu, the completion system inserts the method call in the code

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("");
    for(int i=0;i<remainder;i++){
        padding.append("");
    }
    input = input+padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```


COMPONENTS

- Trigger
- Algorithm
- Context
- Presentation

Trigger:
Typing the dot (.) after the padding variable activates the code completion system). Dot is the trigger character here.

Algorithm:
All methods of StringBuffer receiver type are collected and sorted alphabetically. This constitutes the algorithm

Presentation:
1. a popup menu with method names
2. Another window that shows documentation of the selected method

Code Context:
Type of the padding variable

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append(" ");
    for(int i=0;i<remainder;i++){
        padding.append(" ");
    }
    input = input+padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```

append
public StringBuffer append(boolean b)

Appends the string representation of the boolean argument to the sequence.
The overall effect is exactly as if the argument were converted to a string by the method `String.valueOf(boolean)`, and the characters of that string were then appended to this character sequence.

Parameters:
b - a boolean.

Returns:
a reference to this object

Press 'Tab' from proposal table or click for focus

Target: Method Call
When a developer selects a completion proposal from the popup menu, the completion system inserts the method call in the code

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append(" ");
    for(int i=0;i<remainder;i++){
        padding.append(" ");
    }
    input = input+padding.toString();
}
for(int i=0, count =1;i+this.shingleSize<input.length();i++,count++){
    hmTokenList.put(input.substring(i,i+this.shingleSize),count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```



COMPONENTS

- Trigger
- Algorithm
- Context
- Presentation
- Target

Trigger:
Typing the dot (.) after the padding variable activates the code completion system). Dot is the trigger character here.

Algorithm:
All methods of StringBuffer receiver type are collected and sorted alphabetically. This constitutes the algorithm

Presentation:
1. a popup menu with method names
2. Another window that shows documentation of the selected method

Code Context:
Type of the padding variable

```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("_");
    for(int i=0; i<remainder; i++){
        padding.append("_");
    }
    input = input+ padding.toString();
}
for(int i=0, count =1; i+this.shingleSize<input.length(); i++,count++){
    hmTokenList.put(input.substring(i, i+this.shingleSize), count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```

append
public StringBuffer append(boolean b)
Appends the string representation of the boolean argument to the sequence.
The overall effect is exactly as if the argument were converted to a string by the method `String.valueOf(boolean)`, and the characters of that string were then appended to this character sequence.
Parameters:
b - a boolean.
Returns:
a reference to this object.
Press 'Tab' from proposal table or click for focus

Target: Method Call
When a developer selects a completion proposal from the popup menu, the completion system inserts the method call in the code

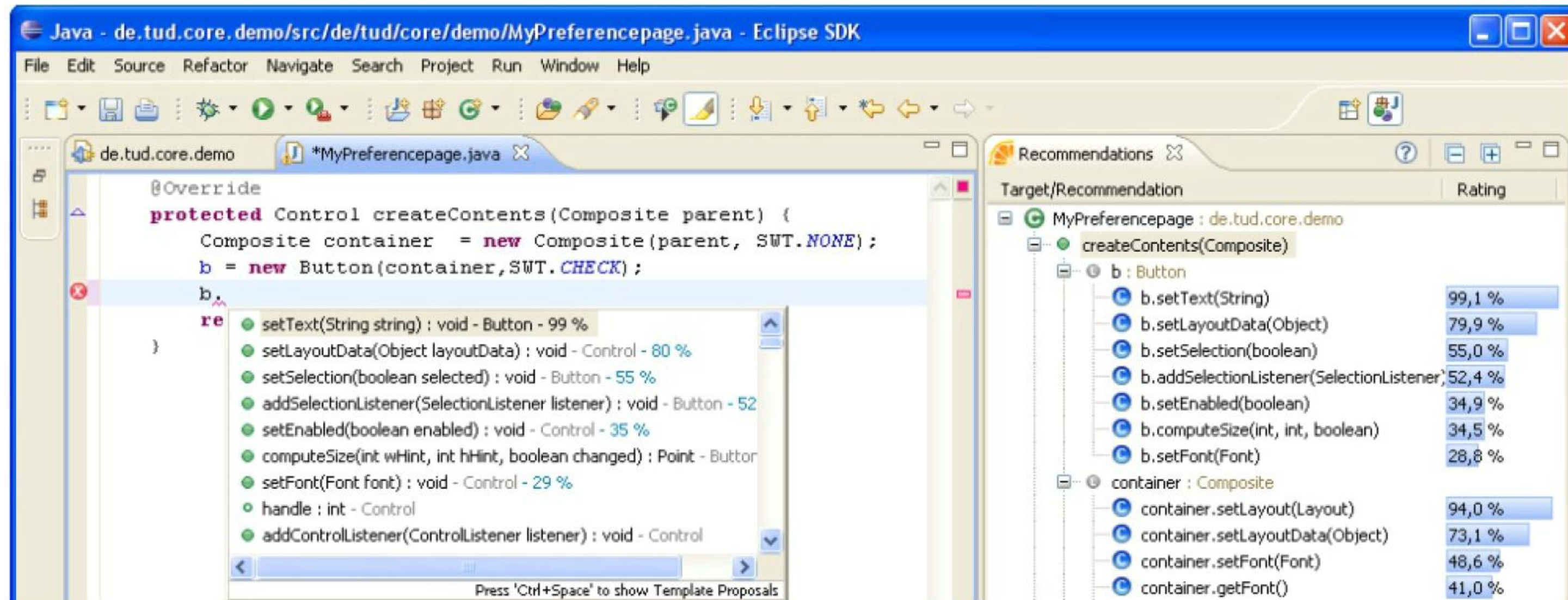
```
if(input.length()%this.shingleSize!=0){
    //add padding
    int remainder = input.length()%this.shingleSize;
    StringBuffer padding = new StringBuffer();
    padding.append("_");
    for(int i=0; i<remainder; i++){
        padding.append("_");
    }
    input = input+ padding.toString();
}
for(int i=0, count =1; i+this.shingleSize<input.length(); i++,count++){
    hmTokenList.put(input.substring(i, i+this.shingleSize), count);
}

//System.out.println(" Shingles : "+hmTokenList.keySet());
return new ArrayList(hmTokenList.keySet());
}

@Override
public Set<String> tokenizeToSet(String arg0) {
    // TODO Auto-generated method stub
    return null;
}
```


METHOD CALL COMPLETION

- Focus on completing method calls



Bruch et al., FSE, 2009

METHOD PARAMETER COMPLETION

- Focus on completing method parameters
- The term parameter refers to actual parameter (or argument) of the call

The diagram illustrates the process of method parameter completion in three stages, labeled A, B, and C, showing how an IDE suggests parameters for the `content.add` method.

Stage A: The code snippet shows the initialization of an `action` and its addition to a group `grp`. The `content.add` method is called with two arguments: `BorderLayout.SOUTH` and `action`. A yellow box labeled **A** highlights the `content.add` method call.

Stage B: The code snippet is the same as in Stage A. A yellow box labeled **B** highlights the `content.add` method call. A dropdown menu is shown, suggesting the first parameter (the layout) with options: `BorderLayout.SOUTH`, `BorderLayout.NORTH`, `BorderLayout.WEST`, `BorderLayout.EAST`, `BorderLayout.CENTER`, and `CONTEXT_ADD_DIALOG_LAST_SELECTION`.

Stage C: The code snippet is the same as in Stage A. A yellow box labeled **C** highlights the `content.add` method call. A dropdown menu is shown, suggesting the second parameter (the component) with options: `action`, `separator`, `typePanel`, `GUIUtilities.createMultilineLabel()`, `comp`, `content`, `combo`, `cancel`, `list`, `ok`, and `rootPane`.

Asaduzzaman et al., ICSME, 2015

METHOD BODY COMPLETION

- Given a partially written method body, these techniques focus on completing the remaining part of the code.

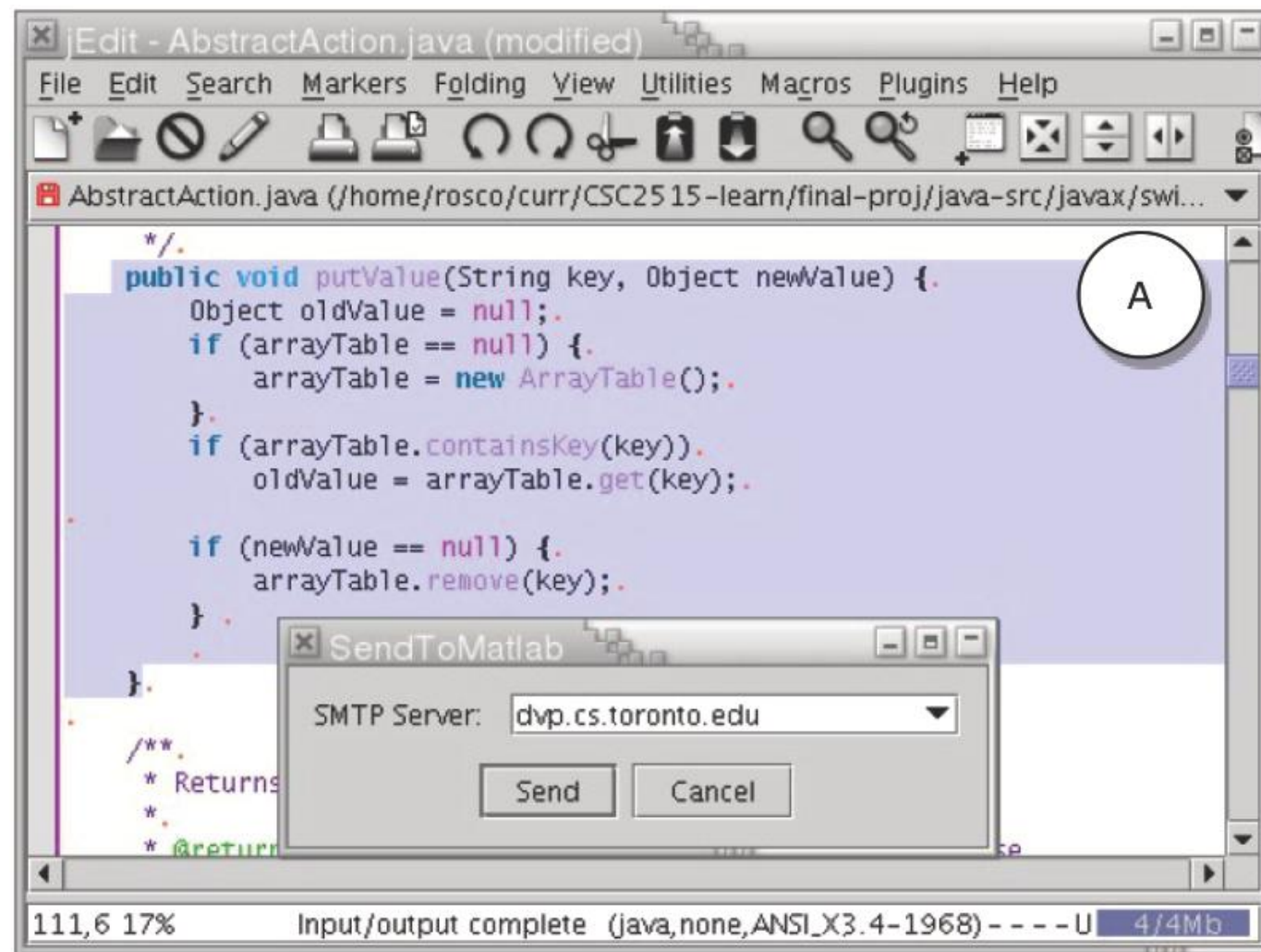


Figure A shows an IDE window titled "Edit - AbstractAction.java (modified)". The code editor displays a Java method `putValue` with a partially completed body. A dialog box titled "SendToMatlab" is open in the foreground, showing an SMTP Server dropdown set to "dvp.cs.toronto.edu" and "Send" and "Cancel" buttons. The status bar at the bottom indicates "111,6 17% Input/output complete (java,none,ANSI_X3.4-1968) - - - - U 4/4Mb".

```
public void putValue(String key, Object newValue) {  
    Object oldValue = null;  
    if (arrayTable == null) {  
        arrayTable = new ArrayTable();  
    }  
    if (arrayTable.containsKey(key)) {  
        oldValue = arrayTable.get(key);  
    }  
    if (newValue == null) {  
        arrayTable.remove(key);  
    }  
}
```

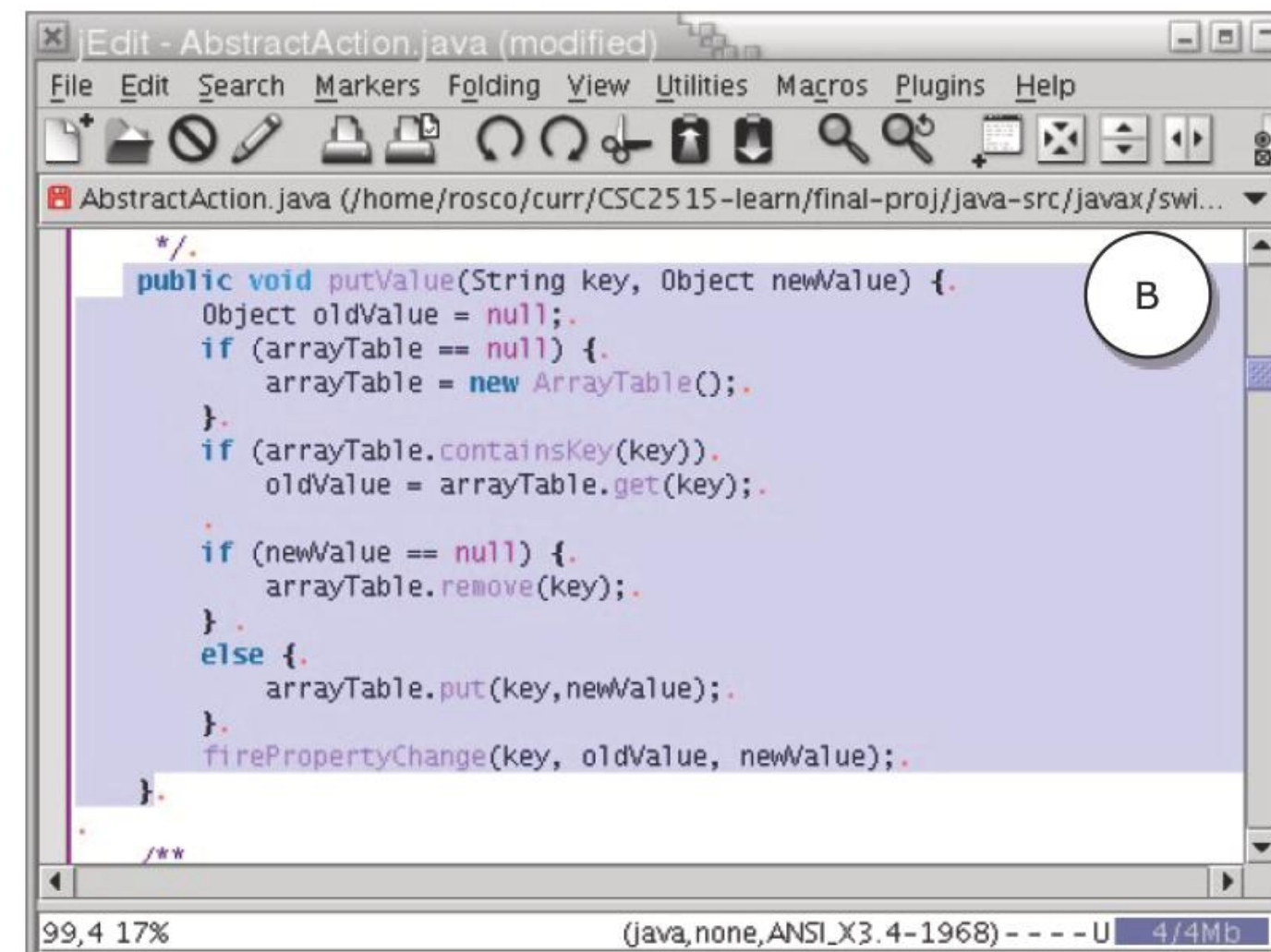


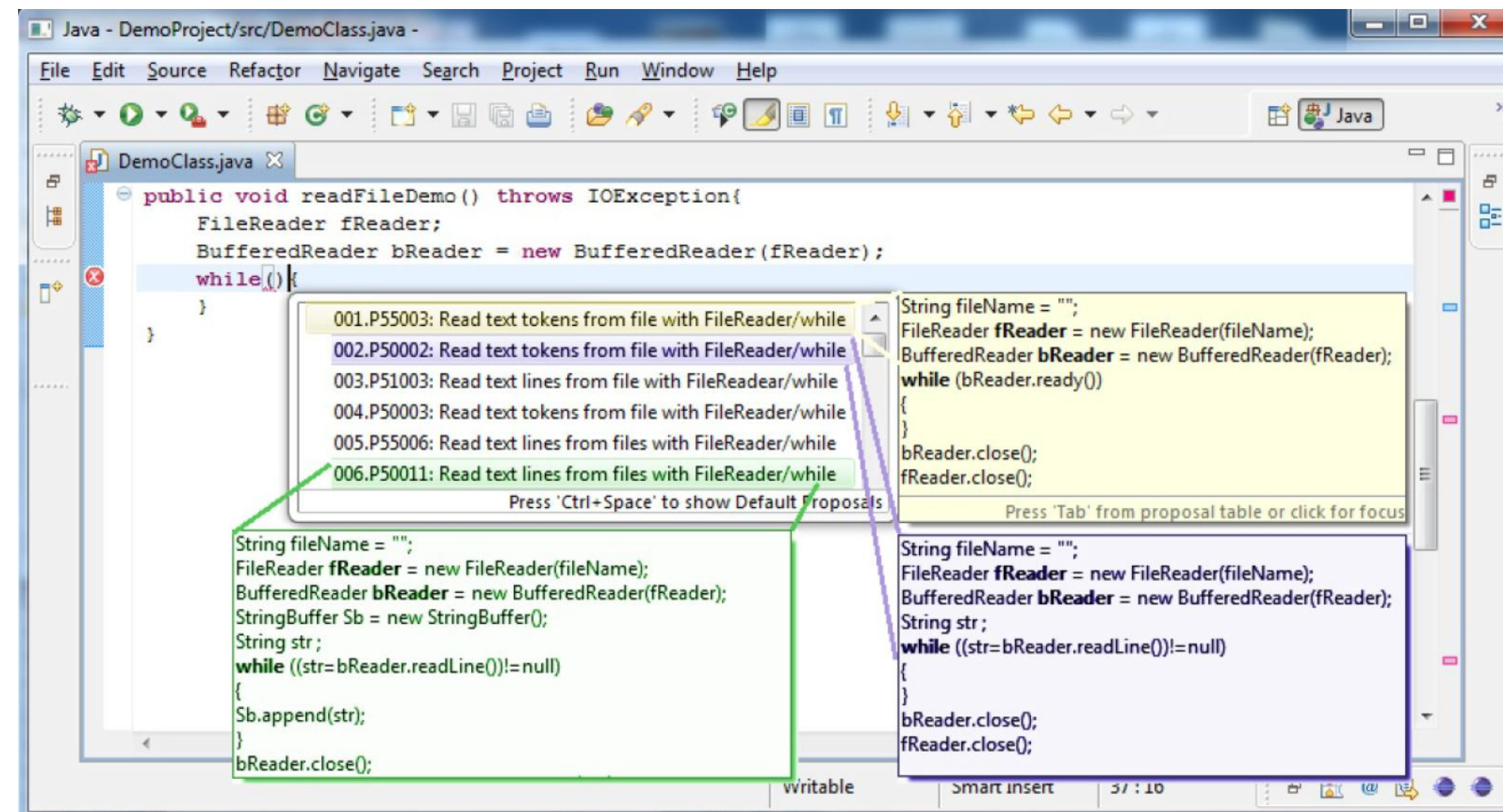
Figure B shows the same IDE window as Figure A, but with the "SendToMatlab" dialog box closed. The code editor now shows the completed method body. The status bar at the bottom indicates "99,4 17% (java,none,ANSI_X3.4-1968) - - - - U 4/4Mb".

```
public void putValue(String key, Object newValue) {  
    Object oldValue = null;  
    if (arrayTable == null) {  
        arrayTable = new ArrayTable();  
    }  
    if (arrayTable.containsKey(key)) {  
        oldValue = arrayTable.get(key);  
    }  
    if (newValue == null) {  
        arrayTable.remove(key);  
    }  
    else {  
        arrayTable.put(key, newValue);  
    }  
    firePropertyChange(key, oldValue, newValue);  
}
```

Hill and Rideout, ASE, 2004

API USAGE PATTERN COMPLETION

- Targets completing multiple lines of code
- Based on mining API usage patterns

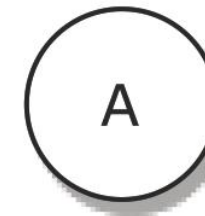


Nguyen et al., ICSE, 2012

KEYWORD-BASED CODE COMPLETION

- Targets completing an expression

```
public List<String> getLines(BufferedReader in) throws Exception {  
    List<String> lines = new Vector<String>();  
    while (in.ready()) {  
        add line|  
    }  
    return lines;  
}
```



```
public List<String> getLines(BufferedReader in) throws Exception {  
    List<String> lines = new Vector<String>();  
    while (in.ready()) {  
        lines.add(in.readLine());|  
    }  
    return lines;  
}
```

Little and Miller., ASE, 2007

OTHER FORMS OF CODE COMPLETION

- Improve code completion menu
- Complete language constructs other than those discussed earlier
- Help Discovering APIs

```
public static void main(String[] args) {  
    Pattern p =  
}
```

☐ Ignore Case

(enter a positive test case above)

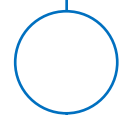
(enter a negative test case above)

Omar et al., VLHCC, 2011

Part II

Context-Sensitive Code Completion tool (CSCC)





Motivation



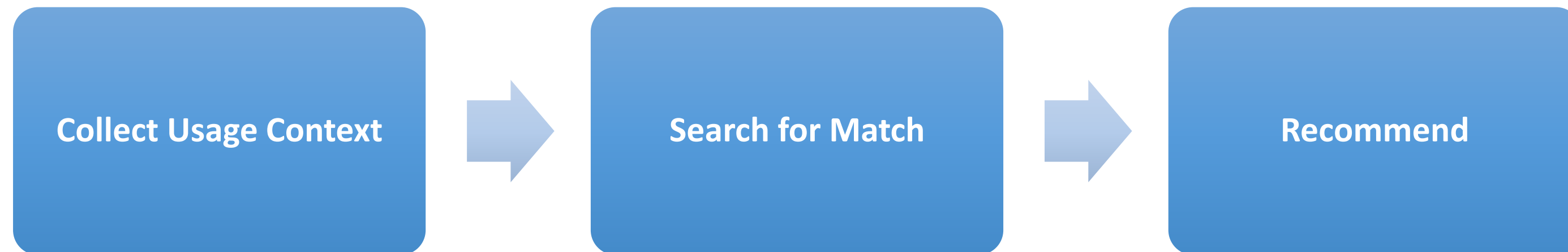
University of Windsor

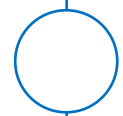
MOTIVATION OF CSCC: A CONTEXT-SENSITIVE CODE COMPLETION SYSTEM

```
BufferedReader br;  
String line;  
StringBuffer sb = new StringBuilder();  
br= new BufferedReader(new  
    FileReader("file.txt"));  
try{  
    while ((line = br.readLine()) != null){  
        sb . append ( l i n e ) ;  
        sb . append ( '\n ' ) ;  
    }  
}finally {br . close ();}
```



PROPOSED TECHNIQUE





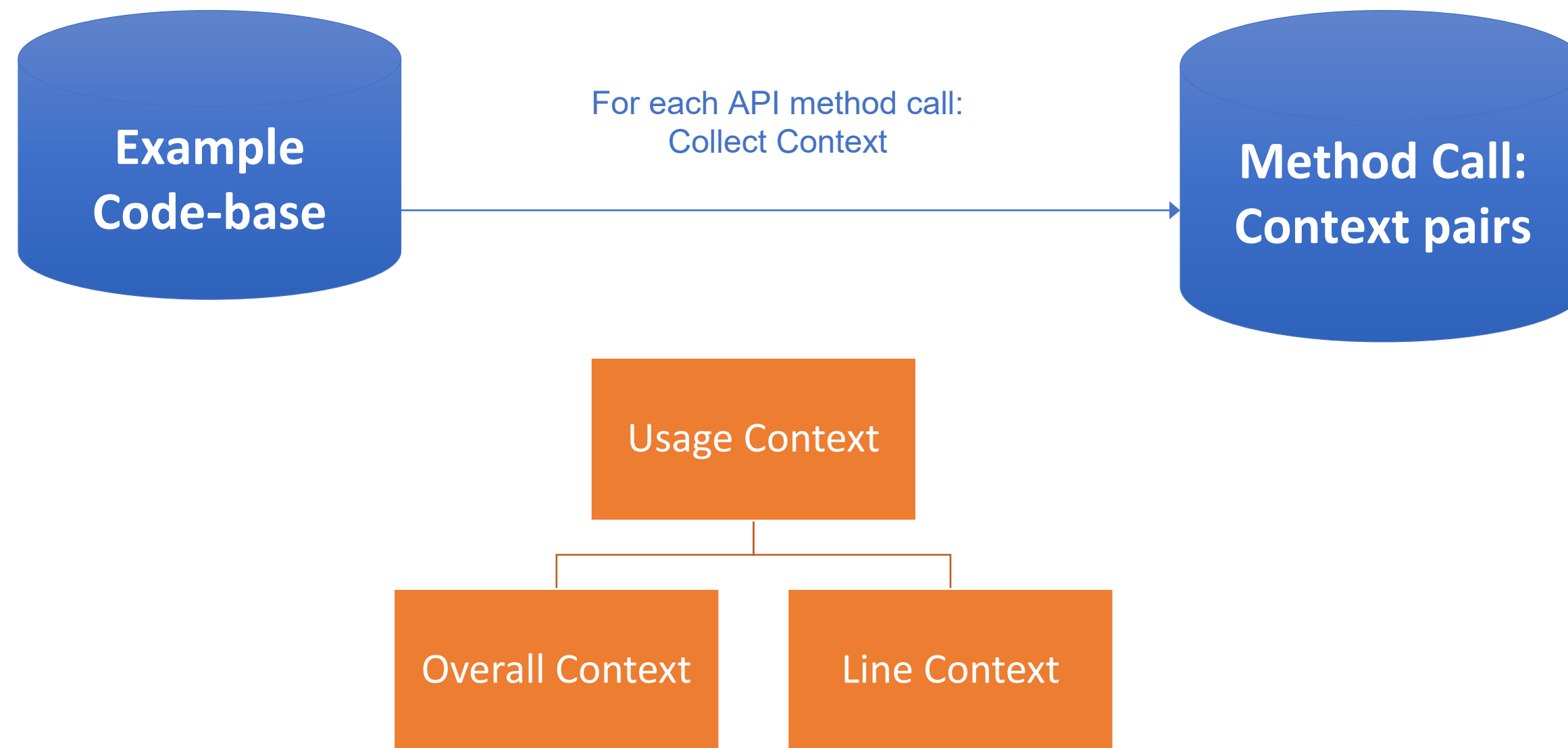
Motivation



University of Windsor

Collect Usage Context

- Step-1: Collect usage context of API method calls



Overall Context

...

```
10. getButton.setEnabled(false);}
```

```
11. void create (Composite parent){
```

```
12. Text text = new Text(parent, SWT.MULTI);
```

```
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
```

...



API Method Call

Overall Context

...

```
10. getButton.setEnabled(false);
```

```
11. void create (Composite parent){
```

```
12. Text text = new Text(parent, SWT.MULTI);
```

```
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
```

...

Only Consider
Top-4 lines

API Method Call

Overall Context

Only Consider
Top-4 lines

```
10. getButton.setEnabled(false);}
11. void create (Composite parent){
12. Text text = new Text (parent,SWT.MULTI);
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
...
```

API Method Call

1. Class or interface name: **Composite**, **Text**, **SWT**, **JFaceColors**

Overall Context

Only Consider
Top-4 lines

```
10. getButton.setEnabled(false);}
11. void create (Composite parent){
12. Text text = new Text(parent,SWT.MULTI);
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
...
```

API Method Call

1. Any class or interface name: Composite, Text, SWT, JFaceColors
2. Any Method name: **Text**, **setForeground**, **getErrorText**

Overall Context

Only Consider
Top-4 lines

```
10. getButton.setEnabled(false);}
11. void create (Composite parent){
12. Text text = new Text(parent,SWT.MULTI);
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
...
```

API Method Call

1. Any class or interface name: Composite, Text, SWT, JFaceColors
2. Any Method name: Text, setForeground, getErrorText
3. Any Java keyword: **void**, **new**

Overall Context

Only Consider
Top-4 lines

```
10. getButton.setEnabled(false);}
11. void create (Composite parent){
12. Text text = new Text(parent,SWT.MULTI);
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
...
```

API Method Call

1. Any class or interface name: Composite, Text, SWT, JFaceColors
2. Any Method name: Text, setForeground, getErrorText
3. Any Java keyword: void, new, void, Composite, Text, new, Text, SWT, setForeground

Line Context

```
10. getButton.setEnabled(false);}
11. void create (Composite parent){
12. Text text = new Text(parent,SWT.MULTI);
13. text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
...
```

Consider the
line where
method call
appears

API Method Call

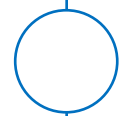
Line Context

Consider the
line where
method call
appears

```
10. getButton.setEnabled(false);}
11. void create (Composite parent){
12. Text text = new Text(parent,SWT.MULTI);
13.
text.setForeground(JFaceColors.getErrorText(text.getDisplay()));
...
```

API Method Call

1. Any class or interface name: Composite, Text, SWT, JFaceColors
2. Any Method name: Text, setForeground, getErrorText
3. Any Java keyword: void, new, void, Composite, Text, new, Text, SWT, setForeground, JFaceColors, getErrorText



Storing Data



University of Windsor

How we store the data?

- We use a two-level indexing scheme to organize the collected usage patterns or examples
- At the first level we use type name of objects as an index
- At the second level we use an inverted index structure



PROPOSED TECHNIQUE

- The user invokes the method completion request:

```
10. JInternalFrame myFrame =  
    new JInternalFrame(); →  
11. myFrame.setVisible(true);  
12  
13. myDesktop.add(myFrame);  
14. myFrame
```

request for a method
call completion

PROPOSED TECHNIQUE

- The user invokes the method completion request

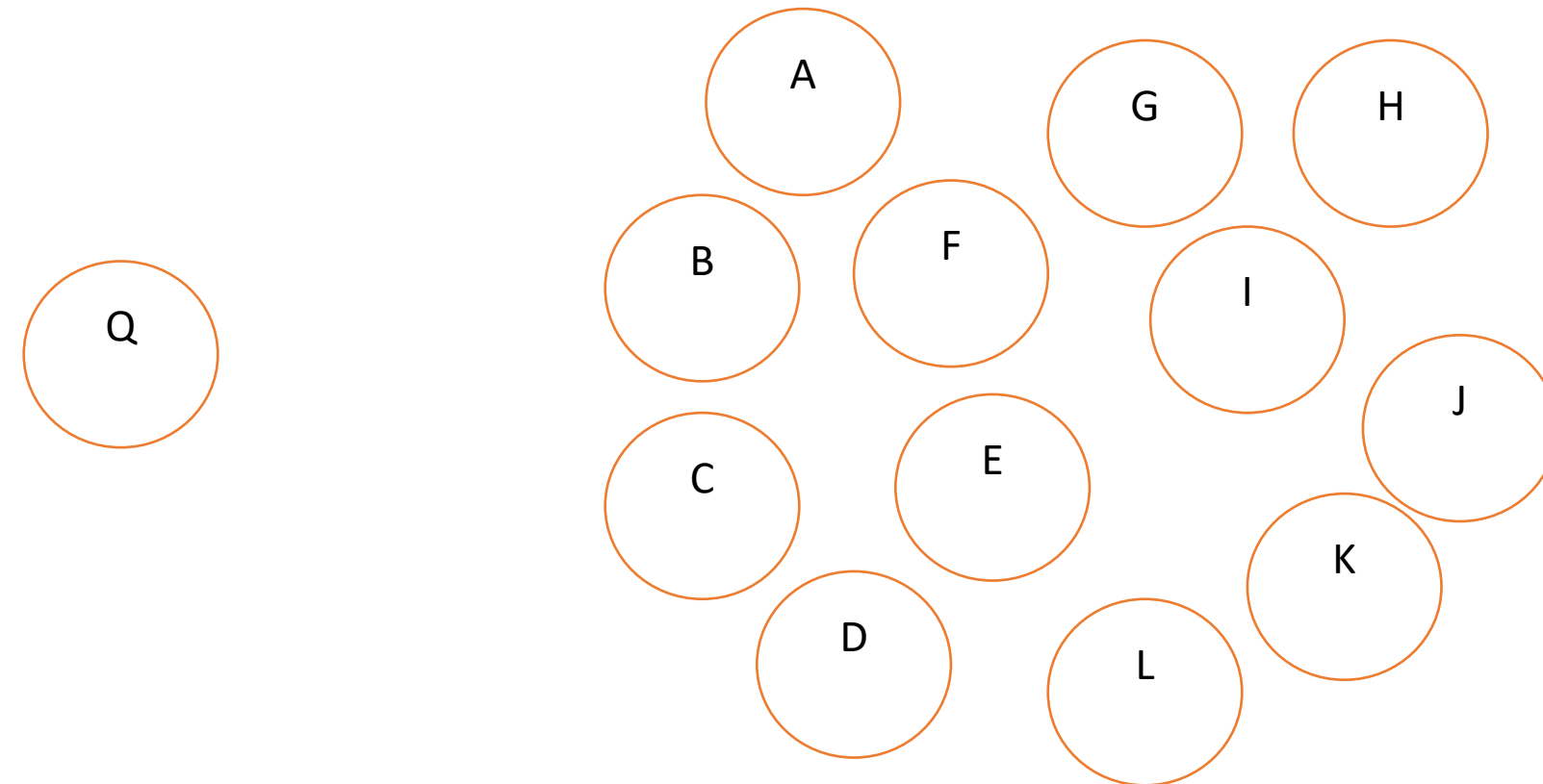
```
10. JInternalFrame myFrame =  
    new JInternalFrame(); →  
11. myFrame.setVisible(true);  
12.  
13. myDesktop.add(myFrame);  
14. myFrame
```

request for a method
call completion

- Identify receiver type: `javax.swing.JFrame`
- Determine query contexts
Overall Context: `JInternalFrame`, `new`, `JInternalFrame`, `setVisible`,
`add`
- Line Context: Empty

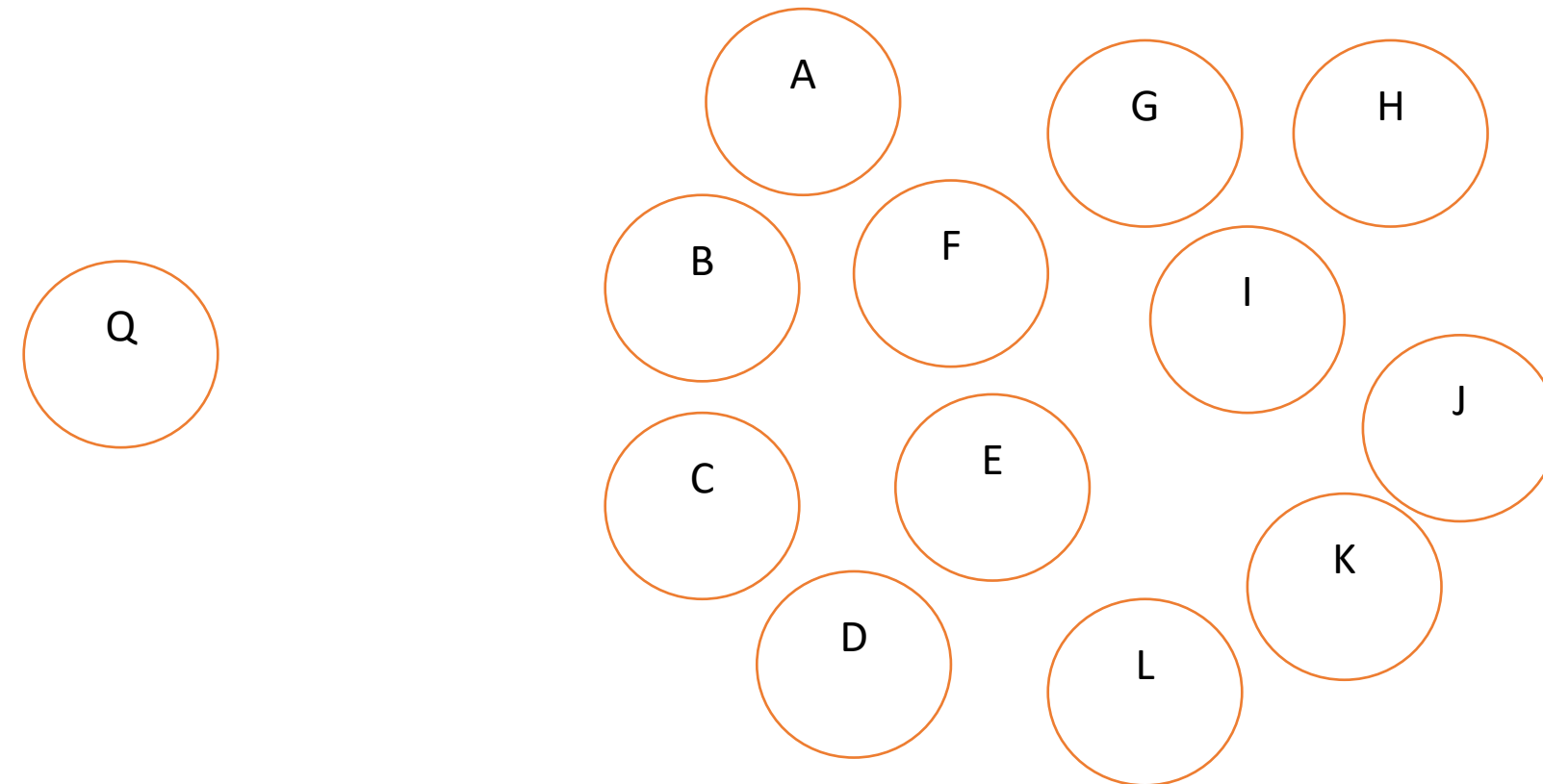
Step 2: Find Matching

- Step 2: Look for a match between query context and API method call context



Step 2: Find Matching

- Step 2: Look for a match between query context and API method call context



- We apply **SimHashing** to select a small set of candidates

SimHashing

- To speed-up mapping process we use the simhash technique. Simhash gives a small binary representation of a large string.
- We generate simhash value for both context and content.

A. Query Context (num1>num2)	100010011011010100000000000001011
B. A Method Call Context: From code example	11100000101001010110000011001011

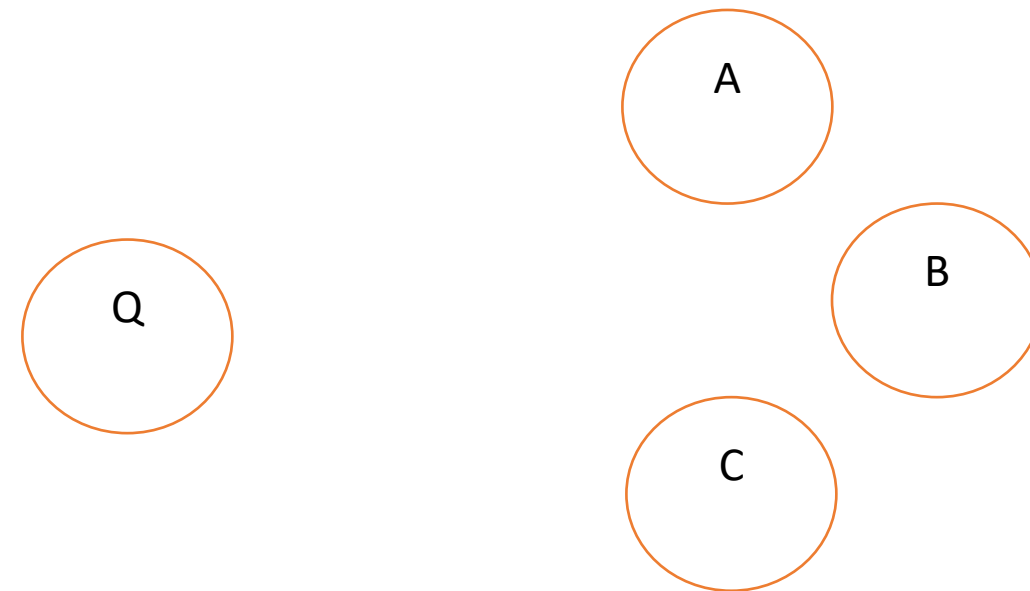
- Determine hamming distance between two simhash values.

Hamming Distance Between A & B: 9

- We select 400 method call examples that gives lowest hamming distance

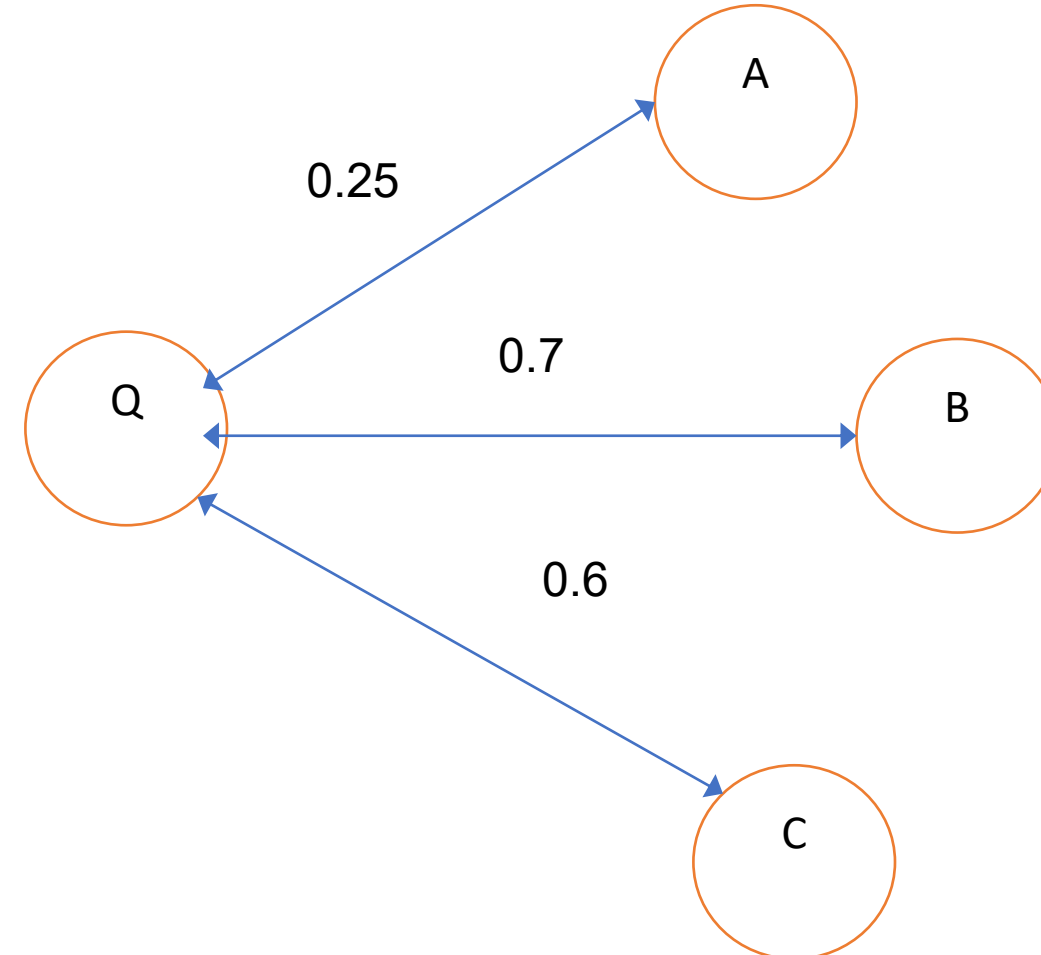
Step 2: Find Matching

- We select a small set of method call-context pairs



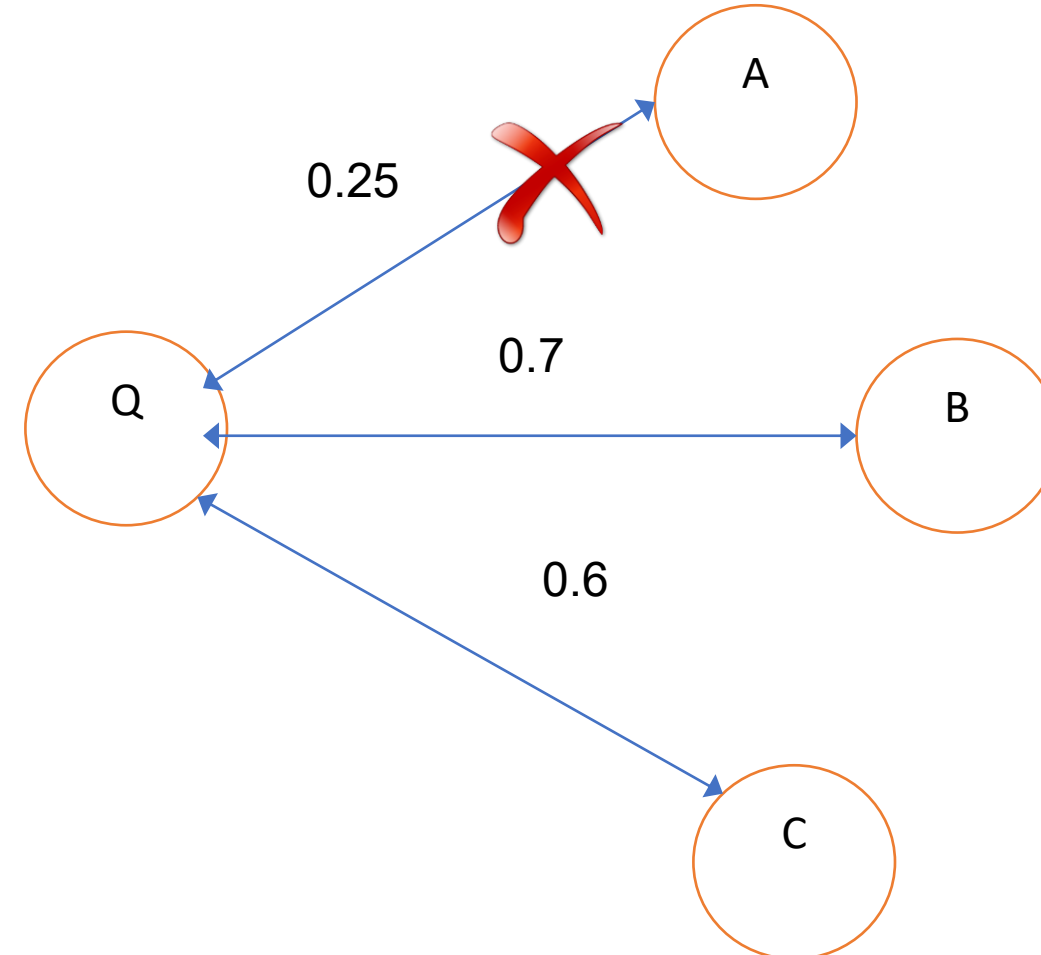
Step 2: Find Matching

- We now apply string similarity distance to measure similarity



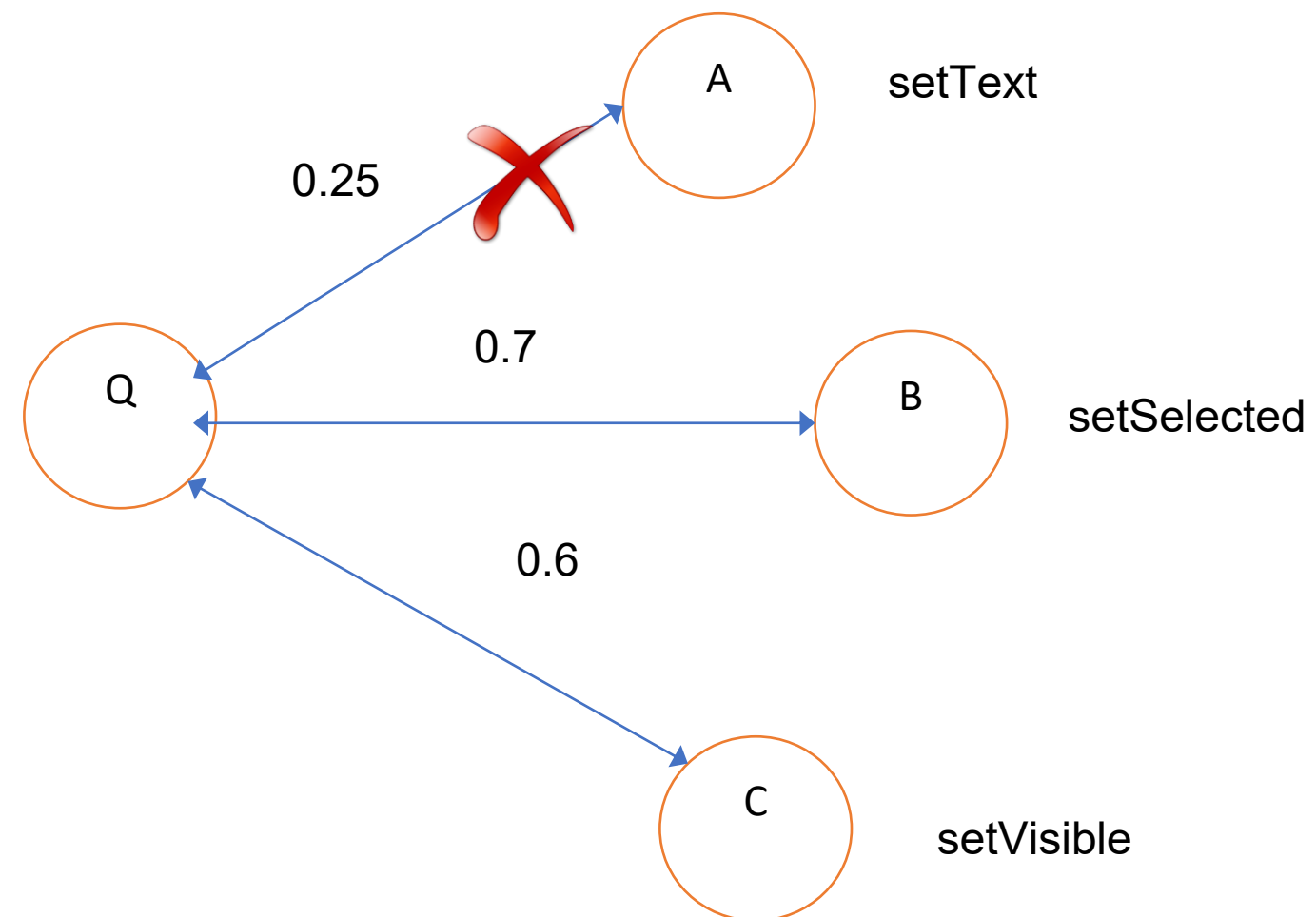
Step 2: Find Matching

- We now apply string similarity distance to measure similarity

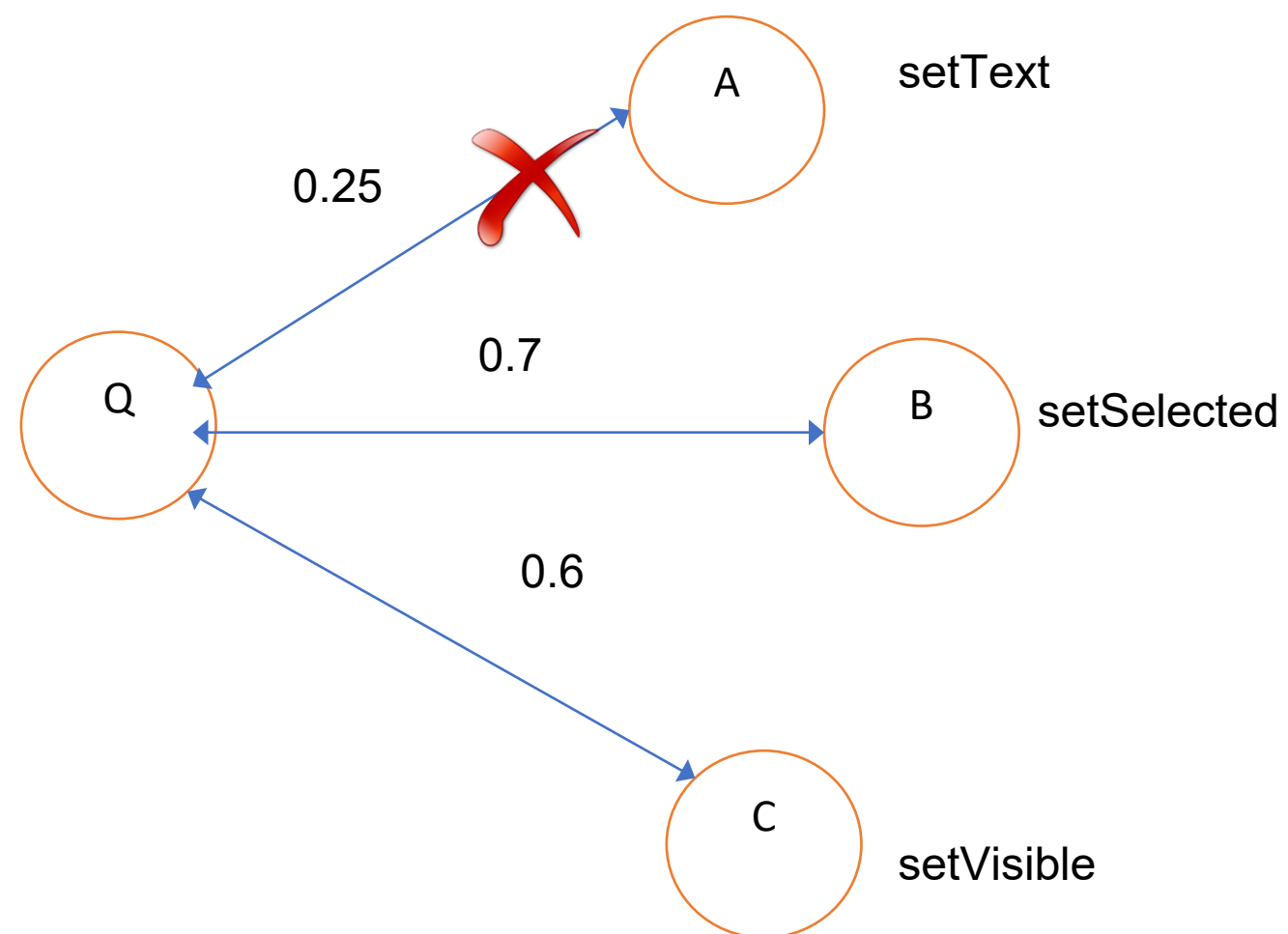


Step 2: Find Matching

- We now apply string similarity distance to measure similarity



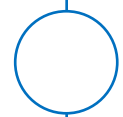
Step 3: Recommend Method Call



```
10. JInternalFrame myFrame =  
    new JInternalFrame();  
11. myFrame.setVisible(true);  
12  
13. myDesktop.add(myFrame);  
14. myFrame
```

request for a method
call completion

1. Recommendations:
setSelected
setVisible



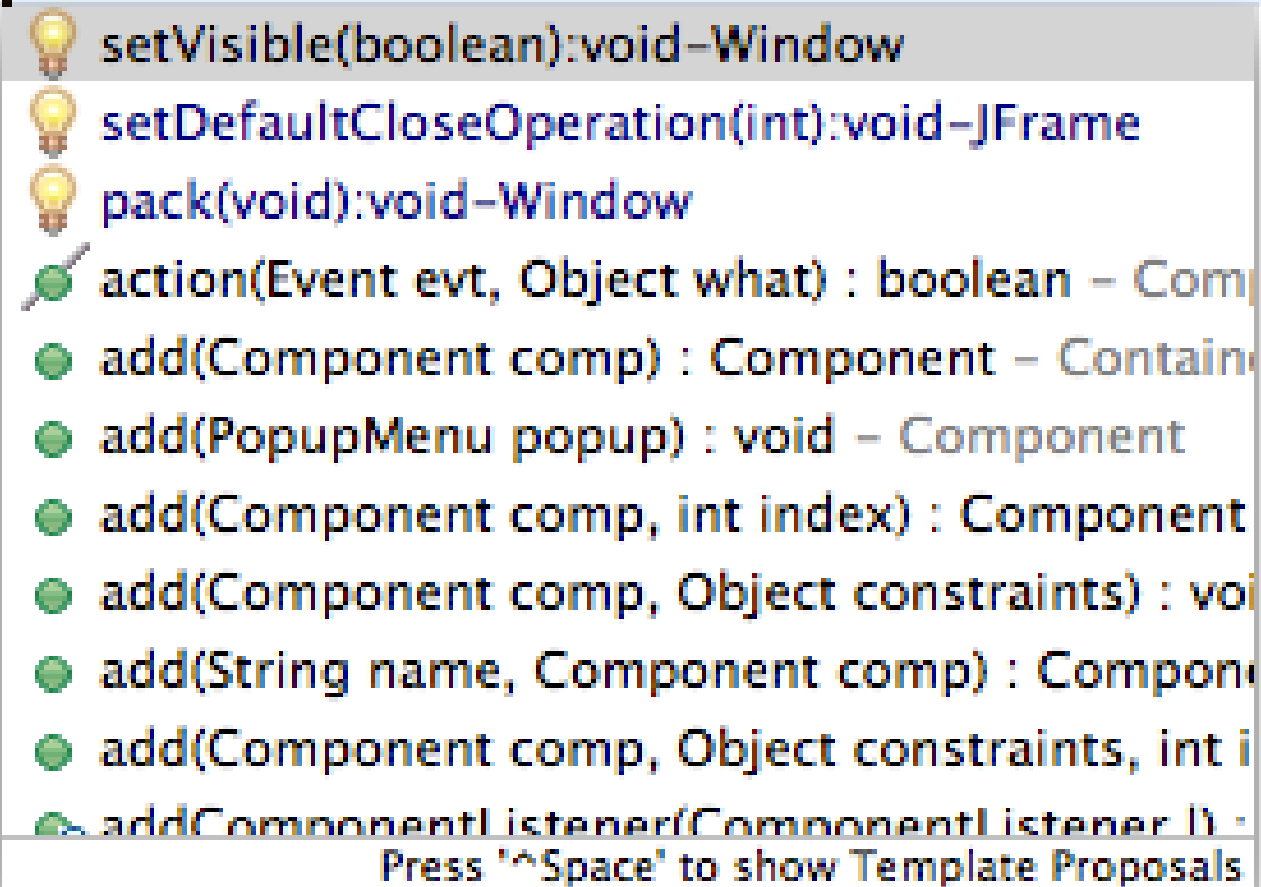
Example



University of Windsor

Example of Recommendation

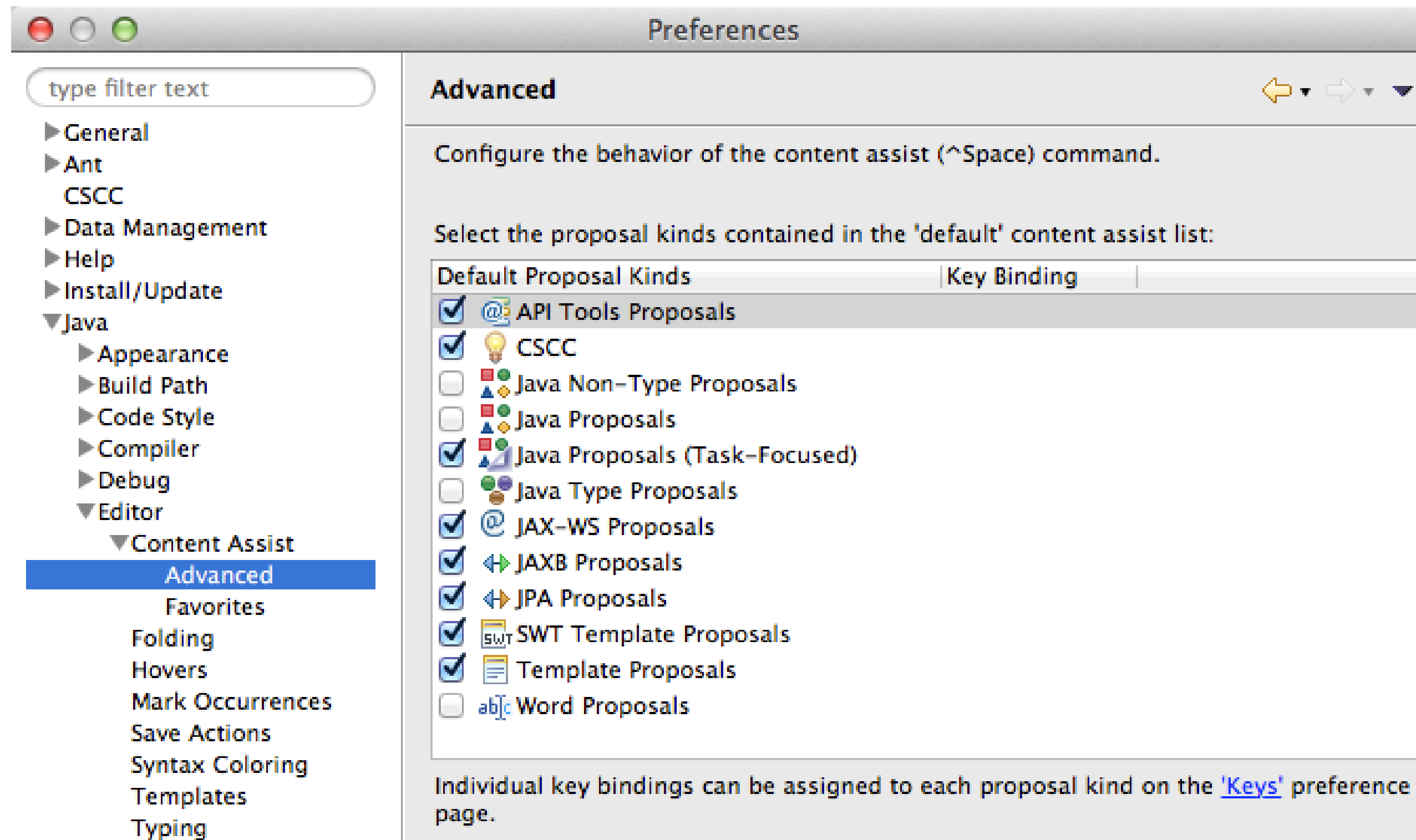
```
    JFrame frame = new JFrame();  
    Container c = frame.getContentPane();  
    frame.add(new JLabel("label"));  
    frame.  
    }  
}
```



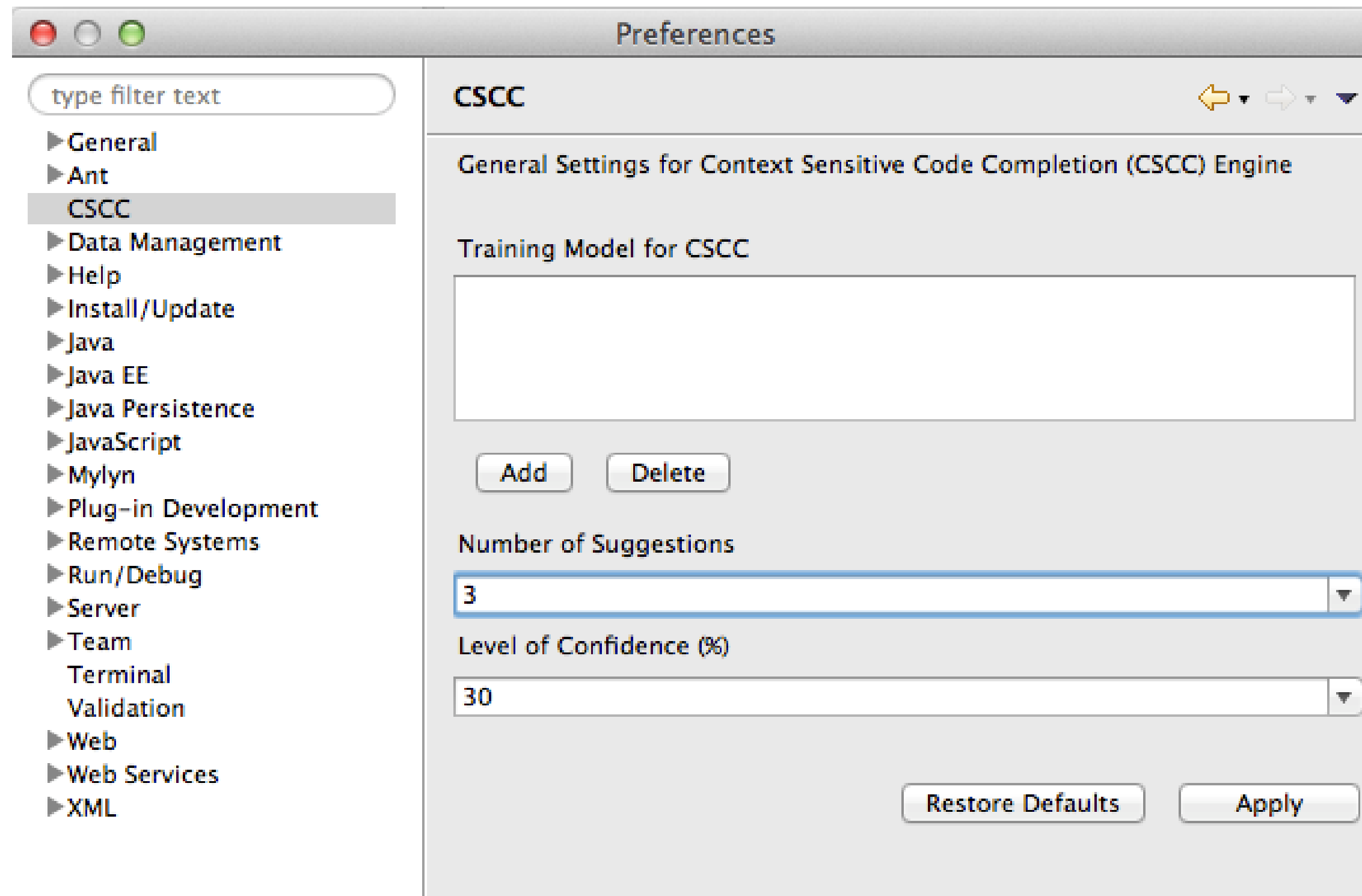
- setVisible(boolean):void-Window
- setDefaultCloseOperation(int):void-JFrame
- pack(void):void-Window
- action(Event evt, Object what) : boolean - Component
- add(Component comp) : Component - Container
- add(PopupMenu popup) : void - Component
- add(Component comp, int index) : Component
- add(Component comp, Object constraints) : void
- add(String name, Component comp) : Component
- add(Component comp, Object constraints, int index) : Component
- addComponentListener(ComponentListener l) : void

Press '^Space' to show Template Proposals

Configuration Option



Configuration Option



User Study

- Use simple observation process
- Tasks: developing graphical user interfaces
- Conduct semi-structured interview to gain more insights
- Use Eclipse IDE and Java Swing/AWT libraries



Lessons Learned

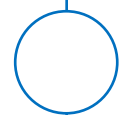
- In general, recommendations were useful
- The tool made recommendations reasonably fast



Part III

Conclusions





Lessons Learned



Lessons Learned

- What if line context is empty and the overall context differs a lot ?



Lessons Learned

- Users ask support for instantiating object:
`Toolkit.getDefaultToolkit.getImage()`

Obtaining the tool

- **CSCC (@ Dr. Muhammad Asaduzzaman)** is available as an Eclipse plugin.
- The tool can be downloaded from this link:
https://dl.dropboxusercontent.com/u/5512193/CSCC/CSCCPlugin_1.0.0.201407161243.jar
- More information about the tool can be found:
<http://asaduzzamanparvez.wordpress.com/csc/>
- The tool requires Java runtime environment and Eclipse Kepler version.



Benefits of Using Simhash Technique

- The tool uses Simhashing to quickly look for method calls whose context matches with query context



How much benefit
we gain?

Benefits of Using Simhash Technique

- Run experiment on Eclipse system and SWT library methods.
- Test: 4500 queries
- Training: 40,863 method calls
- Calculate average time to recommend method calls



Benefits of Using Simhash Technique

- Run experiment on Eclipse system and SWT library methods.
- Test: 4500 queries
- Training: 40,863 method calls
- Calculate average time to recommend method calls

Using Simhashing	Without using Simhashing
1.94 ms	50 ms

Simhashing makes the tool more than 25 times faster

Conclusion

- The tool only requires tokenization.
- Can be easily extended for other programming languages.
- We are planning to conduct more extensive user study to understand the usefulness and limitations of the tool.



Part IV

What's Next



Next Lecture



University of Windsor

Next lecture

- Next lecture includes Guided Learning:
 - Code Completion Tools



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor